

COMPLETE PRESENTATION STUDY GUIDE

Tamil and Bangla Text Classification - Cross-Language Indian NLP Study

1. PROJECT TITLE

"A Comprehensive Evaluation of Machine Learning, Deep Learning, and Transformer Models for Tamil and Bangla Text Classification: A Cross-Language Indian NLP Study"

Authors: Kratika Rathi, Gauri Khandelwal

Guide: Dr. Ringki Das

University: GLA University, Mathura, India

Department: Computer Science and Engineering

2. WHAT IS THIS PROJECT ABOUT? (One-line answer)

We tested 20 different AI models on Tamil and Bangla languages to find which model works best for classifying news text in Indian languages.

3. MOTIVATION / WHY WE DID THIS

- > India has 22 official languages but most NLP research is only for English
 - > Tamil (80M speakers) and Bangla (230M speakers) are major languages with very less research
 - > No existing work compares 20 models across ALL paradigms for Indian languages
 - > Developers don't know which model to pick for Tamil/Bangla - we give clear answers
-

4. PROBLEM STATEMENT

Given news text in Tamil or Bangla, automatically classify it into the correct news category. We want to find:

1. Which model gives best accuracy?
 2. Does the same model work for both languages?
 3. What is the trade-off between accuracy and training time?
 4. Do expensive models (LLMs, Transformers) always beat simple models (SVM)?
-

5. LANGUAGES STUDIED

Property	Tamil	Bangla
Family	Dravidian	Indo-Aryan
Type	Agglutinative (words = root + suffi	Fusional (words change form)
Speakers	80 million	230 million
Script	Tamil script	Bengali script
Countries	India, Sri Lanka, Singapore	India, Bangladesh

Key Difference: Tamil joins morphemes to make words (agglutinative). Bangla changes word forms (fusional). This affects which models work best.

6. DATASETS USED

Tamil Dataset

- > Source: IndicGLUE (indic_glue, inltk.ta) from Hugging Face
- > Total Samples: 5,346
- > Classes: 3 (news categories, original labels [1, 5, 6] remapped to [0, 1, 2])
- > Distribution: Class 0: 1,567 | Class 1: 1,519 | Class 2: 2,260
- > Split: 80% train (4,276) / 20% test (1,070)
- > Type: News headlines

Bangla Dataset

- > Source: AI4Bharat IndicNLP News Articles
- > Total Samples: 20,000
- > Classes: 6+ (News, Sports, Technology, Politics, Entertainment, Business)
- > Split: 80% train (16,000) / 20% test (4,000)
- > Type: Full news articles

7. ALL 20 MODELS (5 Categories)

Category 1: Traditional Machine Learning (4 models)

Model	How it works
Naive Bayes	Uses probability - assumes features
SVM (RBF kernel)	Finds optimal boundary between clas
Logistic Regression	Linear model with sigmoid function
Random Forest	100 decision trees vote together

Category 2: Ensemble Learning (1 model)

Model	How it works
XGBoost	Builds trees one by one, each fixin

Category 3: Deep Learning (5 models)

Model	How it works
CNN	Convolution filters detect local pa
LSTM	Remembers long sequences using gate
Bi-LSTM	LSTM in both forward + backward dir
GRU	Simpler version of LSTM (2 gates in
CNN-LSTM	CNN extracts local features, then L

Category 4: Transformers (5 models)

Model	How it works
BERT (Multilingual)	Pre-trained on 104 languages using

XLM-RoBERTa	Trained on 2.5TB data in 100 languages
IndicBERT v2	AI4Bharat model trained specifically
MuRIL	Google model for 17 Indian languages
RemBERT	Large multilingual model with 559M

Category 5: Large Language Models (5 models)

Model	How it works
IndicGemma	Google Gemma adapted for Indian languages
Llama 3.1	Meta's 8B parameter open-source LLM
Qwen 2.5	Alibaba's multilingual LLM
Sarban 1	Indic-focused LLM for South Asian languages
M10	Multilingual model supporting 10+ languages

8. METHODOLOGY (How we did it)

Step 1: Data Loading

- > Load Tamil from IndicGLUE via Hugging Face datasets library
- > Load Bangla from AI4Bharat IndicNLP
- > UTF-8 encoding for proper script support

Step 2: Preprocessing

- > Remove empty texts
- > Remap labels to 0-indexed (Tamil: [1,5,6] -> [0,1,2])
- > Stratified train/test split (80/20) with random seed 42

Step 3: Feature Extraction (Two Paths)

Path A - For Traditional ML & Ensemble:

- > TF-IDF Vectorization
- > Character n-grams (1 to 3 characters)
- > Max 10,000 features
- > analyzer='char_wb' (character with word boundaries)
- > sublinear_tf=True (applies log to term frequency)
- > min_df=2, max_df=0.95 (removes very rare and very common terms)

Path B - For Deep Learning:

- > Character-level tokenization
- > Convert to integer sequences
- > Pad to fixed length (200 tokens)
- > Vocabulary size: 10,000

Path C - For Transformers:

- > Subword tokenization using model's own tokenizer
- > Max length: 128 tokens
- > Padding and truncation

Path D - For LLMs:

- > Raw text given as prompt
- > Zero-shot classification (no training)
- > 4-bit NF4 quantization to fit on GPU

Step 4: Training Configuration

Category	Epochs	Batch Size	Optimizer	Learning Rate
Traditional ML	N/A	N/A	Built-in	Built-in
XGBoost	100 trees	N/A	Gradient Boosting	0.1
Deep Learning	5	64	Adam	0.001
Transformers	2	8	AdamW	2e-5
LLMs	0 (zero-shot)	-	-	-

Step 5: Evaluation

- > Accuracy, Precision, Recall, F1-Score (all weighted)
- > Same metrics for all models for fair comparison
- > Random seed = 42 for reproducibility

9. HARDWARE USED

Component	Tamil Evaluation	Bangla Evaluation
GPU	NVIDIA RTX 4070 Laptop (8.6 GB)	NVIDIA RTX 3050 (6 GB)
PyTorch	2.6.0 + CUDA 12.4	2.0+ + CUDA
Python	3.12	3.10+
OS	Windows	Windows

10. RESULTS - TAMIL

Complete Rankings (by F1-Score)

Rank	Model	Category	Accuracy	F1-Score	Time
1	SVM (RBF)	Traditional ML	95.42%	95.43%	39.4s
2	XLM-RoBERTa	Transformer	95.00%	95.01%	262.8s
3	MuRIL	Transformer	95.00%	94.99%	264.8s
4	BERT (Multilingual)	Transformer	94.00%	94.04%	264.5s
5	Logistic Regression	Traditional ML	93.74%	93.75%	1.4s
6	XGBoost	Ensemble	92.43%	92.46%	8.0s
7	Naive Bayes	Traditional ML	91.78%	91.82%	1.1s
8	Random Forest	Traditional ML	87.57%	87.64%	1.7s
9	CNN	Deep Learning	84.11%	84.27%	2.5s
10	Bi-LSTM	Deep Learning	83.83%	83.97%	2.1s
11	LSTM	Deep Learning	42.24%	25.09%	2.0s
12	GRU	Deep Learning	42.24%	25.09%	1.6s
13	CNN-LSTM	Deep Learning	42.24%	25.09%	2.1s
14	M10	LLM	32.20%	22.87%	-
15	Qwen 2.5	LLM	27.00%	21.95%	-

Tamil Champion: SVM (RBF) - 95.43% F1

Tamil Category Averages

- > Traditional ML: 92.16% (best category!)
- > Transformers: 94.68%

- > Deep Learning (converged): 84.12%
- > LLMs: 22.41% (worst - failed)

11. RESULTS - BANGLA

Complete Rankings (by F1-Score)

Rank	Model	Category	Accuracy	F1-Score	Time
1	IndicGemma	LLM	85.43%	84.82%	Pre-trained
2	XLNet	Transformer	82.89%	82.61%	2145.7s
3	Llama 3.1	LLM	82.34%	81.94%	Pre-trained
4	MuRIL	Transformer	81.67%	81.32%	1934.3s
5	IndicBERT v2	Transformer	81.34%	81.00%	1876.2s
6	BERT (Multilingual)	Transformer	80.56%	80.22%	1847.2s
7	Qwen 2.5	LLM	80.12%	79.77%	Pre-trained
8	RemBERT	Transformer	79.23%	78.89%	2234.5s
9	Sarban 1	LLM	78.56%	78.22%	Pre-trained
10	Bi-LSTM	Deep Learning	78.23%	77.89%	678.3s
11	CNN-LSTM	Deep Learning	76.89%	76.56%	589.7s
12	LSTM	Deep Learning	76.54%	76.25%	542.8s
13	M10	LLM	76.54%	76.20%	Pre-trained
14	GRU	Deep Learning	75.67%	75.32%	487.6s
15	CNN	Deep Learning	74.23%	73.89%	398.5s
16	XGBoost	Ensemble	72.34%	72.00%	234.7s
17	Random Forest	Traditional ML	71.56%	71.22%	189.4s
18	SVM (RBF)	Traditional ML	69.87%	69.54%	456.8s
19	Logistic Regression	Traditional ML	69.34%	69.00%	87.3s
20	Naive Bayes	Traditional ML	68.92%	68.57%	12.3s

Bangla Champion: IndicGemma - 84.82% F1

Bangla Category Averages

- > LLMs: 80.19% (best category!)
- > Transformers: 80.81%
- > Deep Learning: 75.98%
- > Traditional ML: 69.58% (worst)

12. KEY CROSS-LANGUAGE COMPARISON

Model	Tamil F1	Bangla F1	Winner
SVM (RBF)	95.43%	69.54%	Tamil (+25.89%)
XLNet	95.01%	82.61%	Tamil (+12.40%)
BERT	94.04%	80.22%	Tamil (+13.82%)
Qwen 2.5	21.95%	79.77%	Bangla (+57.82%)
Naive Bayes	91.82%	68.57%	Tamil (+23.25%)

Shocking finding: Rankings are COMPLETELY OPPOSITE for both languages!

13. WHY RESULTS ARE DIFFERENT - KEY EXPLANATIONS

Why SVM wins for Tamil but not Bangla?

1. Tamil has only 3 clean classes -> easy boundary
2. Character n-grams perfectly capture Tamil agglutinative patterns
3. RBF kernel handles high-dimensional TF-IDF space well
4. Bangla has 6+ overlapping classes -> SVM can't find clean boundaries

Why LLMs win for Bangla but fail for Tamil?

1. LLMs have lots of Bangla in training data (230M speakers, huge internet content)
2. Tamil has less digital presence in LLM training corpora
3. Zero-shot mode needs the model to "understand" the language already
4. 4-bit quantization hurts accuracy for languages with less training

Why LSTM/GRU/CNN-LSTM failed on Tamil (25% F1)?

1. Only 5 training epochs - not enough for RNNs to converge
2. Small dataset (4,276 training samples)
3. These models predict majority class for everything (42.24% accuracy = majority class proportion)
4. Fix: Use 15+ epochs with gradient clipping and LR scheduling
5. CNN and Bi-LSTM worked because: CNN has no sequential dependency, Bi-LSTM has 2x parameters

Why all DL models worked on Bangla?

1. Larger dataset (16,000 training samples) - enough signal
 2. More training time was given
 3. RNNs need minimum 10,000+ samples to learn meaningful patterns
-

14. OVERFITTING ANALYSIS (Important for Viva!)

Is SVM overfitting on Tamil?

Answer: NO, but with caution.

Evidence it's NOT overfitting:

- > 95.43% is on HELD-OUT test set (1,070 unseen samples)
- > Three different transformers also get ~95% (confirms task is easy)
- > Even Naive Bayes gets 91.82% (very simple model)
- > TF-IDF uses min_df=2, max_df=0.95 to prevent memorization

Why scores are so high:

- > Only 3 classes (easy task)
- > Classes are well-separated (different news topics)
- > Clean, professionally curated data
- > Tamil character patterns are very distinctive per topic

What would confirm further:

- > 5-fold cross-validation (gives mean +/- std)
- > Larger dataset with 6+ overlapping classes
- > Learning curves (accuracy vs training size)

What is Zero-Shot Classification?

- > Model classifies text WITHOUT being trained on the specific task
- > We give a prompt like: "Classify this Tamil text into category 0, 1, or 2"
- > Model uses its pre-training knowledge to answer
- > Works well if the model has seen enough of that language during pre-training

What is 4-bit Quantization?

- > Reduces model weights from 32-bit to 4-bit
- > Reduces memory by ~8x (70B model fits in 8GB GPU)
- > Uses NF4 (Normal Float 4-bit) format
- > Trade-off: slight accuracy loss for huge memory savings

16. LIBRARIES AND TOOLS USED

Library	Purpose
PyTorch 2.6.0	Deep learning framework
Scikit-learn	Traditional ML models + metrics
Transformers (HuggingFace)	BERT, XLM-R, MuRIL, etc.
XGBoost	Gradient boosting
Datasets (HuggingFace)	Loading IndicGLUE dataset
BitsAndBytes	4-bit quantization for LLMs
NumPy, Pandas	Data manipulation
CUDA 12.4	GPU acceleration

17. PRACTICAL RECOMMENDATIONS

For Tamil (Dravidian, Agglutinative):

- > Best accuracy: SVM + char n-gram TF-IDF (95.43%, trains in 40 seconds)
- > Fastest: Naive Bayes (91.82%, trains in 1 second)
- > If you need transfer learning: XLM-RoBERTa (95.01%)
- > DO NOT USE: Zero-shot LLMs, under-trained RNNs

For Bangla (Indo-Aryan, Fusional):

- > Best accuracy: IndicGemma (84.82%, no training needed)
- > Best fine-tunable: XLM-RoBERTa (82.61%)
- > India-specific: MuRIL (81.32%)
- > Budget option: Random Forest (71.22%, fast)

General Rule for Indian Languages:

1. Start with SVM + char n-gram TF-IDF as baseline
2. If <=5 clean classes -> ML is likely enough
3. If 6+ overlapping classes -> use Transformers
4. Only use LLMs if language has large digital presence
5. Train RNNs for 15+ epochs minimum

18. LIMITATIONS (Be ready to discuss these)

1. Unequal datasets: Tamil=5,346 samples vs Bangla=20,000 - not fair comparison
 2. Different #classes: Tamil=3 vs Bangla=6+ - inflates Tamil scores
 3. No cross-validation: Single split, no mean+/-std reported
 4. RNNs under-trained: Only 5 epochs for Tamil (should be 15+)
 5. Transformer subset: Only 1,000 samples used for fine-tuning (should use full 4,276)
 6. Only 2 LLMs tested on Tamil: GPU memory issues prevented full LLM evaluation
 7. Single domain (news): May not generalize to social media, reviews, etc.
 8. No statistical significance test: SVM vs XLM-RoBERTa difference (0.42%) may not be significant
 9. Only 2 languages: Need Telugu, Kannada, Hindi, Marathi for stronger claims
 10. SVM may not scale: On larger/harder datasets, transformers will likely win
-

19. FUTURE WORK

1. Run 5-fold cross-validation on all 6,684 Tamil samples
 2. Find larger Tamil dataset (20,000+ samples, 6+ classes)
 3. Fine-tune LLMs on Tamil (compare with zero-shot)
 4. Train LSTM/GRU for 20-50 epochs
 5. Extend to Telugu, Kannada, Malayalam (Dravidian) and Hindi, Marathi (Indo-Aryan)
 6. Test on social media text (code-mixed Tamil-English)
 7. Build ensemble: SVM + XLM-RoBERTa voting
 8. Model compression for mobile deployment
-

20. EXPECTED CROSS QUESTIONS AND ANSWERS

Q1: Why did you choose Tamil and Bangla specifically?

A: Because they come from two completely different language families - Tamil is Dravidian (agglutinative) and Bangla is Indo-Aryan (fusional). This lets us study how language structure affects model performance. Both are major Indian languages with 80M+ and 230M+ speakers respectively.

Q2: Why does SVM beat BERT/Transformers on Tamil?

A: Because the Tamil dataset has only 3 well-separated classes. With clean, distinct categories, character n-gram TF-IDF already captures enough discriminative information. Transformers are designed for complex tasks with millions of samples - on a small clean dataset, their overhead doesn't pay off. The 0.42% difference (SVM: 95.43% vs XLM-R: 95.01%) is minimal.

Q3: Is SVM always better than Transformers for Tamil?

A: NO. Only for simple classification with few clean classes. For harder tasks (sentiment analysis, NER, QA) or datasets with more overlapping classes, transformers will outperform SVM because they understand context and semantics better.

Q4: Why did LSTM and GRU completely fail (25% F1)?

A: Because we only trained them for 5 epochs on 4,276 samples. RNNs learn sequentially and need many iterations. With so few epochs, the loss barely starts decreasing before training ends. They end up predicting the majority class for everything. Fix: train for 15-20 epochs.

Q5: Why CNN and Bi-LSTM worked but not plain LSTM/GRU?

A: CNN captures local patterns immediately through convolution (no sequential dependency). Bi-LSTM has 2x the parameters

(bidirectional = forward + backward), so it learns faster. Plain LSTM/GRU with fewer parameters and one direction need more time to converge.

Q6: Why zero-shot LLMs failed for Tamil (22%)?

A: Three reasons: (1) LLMs have very little Tamil in their pre-training data compared to English/Hindi/Bangla. (2) 4-bit quantization reduces accuracy for under-represented languages. (3) The classification prompt was in English - model may not understand Tamil-to-category mapping well.

Q7: What is the difference between SVM and Logistic Regression?

A: Both are linear classifiers but: LR minimizes log-loss and gives probabilities. SVM maximizes the margin between classes. With RBF kernel, SVM can find non-linear boundaries that LR cannot. That's why SVM (95.43%) beats LR (93.75%).

Q8: Why did you use character n-grams instead of word n-grams?

A: Tamil is agglutinative - one word can have 10+ morphemes joined together, making vocabulary extremely large. Character n-grams capture sub-word patterns that are shared across different word forms. For example, different forms of a verb share character patterns even if the full words are different.

Q9: What is the practical use of this research?

A: (1) News aggregation apps can classify Tamil/Bangla news automatically. (2) Spam detection in regional languages. (3) Content moderation on social media. (4) Government document categorization. (5) Our recommendations help developers choose the right model without wasting time/money.

Q10: How is your work different from existing research?

A: (1) No existing work compares 20 models across ALL 5 paradigms. (2) No existing work does cross-language comparison between Dravidian and Indo-Aryan families. (3) We test LLMs on Tamil for the first time. (4) We give practical deployment guidelines with time/accuracy trade-offs.

Q11: What evaluation metrics did you use and why?

A: Accuracy, Precision, Recall, and F1-Score (weighted). We use WEIGHTED averaging because classes are slightly imbalanced. F1-Score is the primary metric because it balances precision and recall. Accuracy alone can be misleading with imbalanced data.

Q12: What is the significance of the 80/20 split?

A: 80% for training (model learns patterns), 20% for testing (evaluate on unseen data). We use stratified split (same class proportions in both sets) and fixed random seed=42 for reproducibility.

Q13: Can you explain the architecture diagram?

A: Input (Tamil + Bangla corpus) -> Preprocessing (encoding, label mapping, 80/20 split) -> Two parallel feature paths: TF-IDF char n-grams (for ML models) and neural tokenization (for DL/Transformers) -> 20 models in 5 groups -> Evaluation using same metrics -> Cross-language comparison.

Q14: Why not use cross-validation?

A: Time constraint. Each transformer takes 4-5 minutes per run. With 5-fold CV x 5 transformers = 125+ minutes just for transformers. But we acknowledge this as a limitation and recommend it for future work.

Q15: What would happen with a larger Tamil dataset?

A: If Tamil had 20,000 samples with 6+ classes (like Bangla), we expect: (1) SVM would drop to 70-80%. (2) Transformers would become clearly better. (3) LLMs might improve. (4) RNNs would finally converge. The Bangla results already show this pattern.

Q16: What is IndicGLUE?

A: It's the Indian version of GLUE (General Language Understanding Evaluation). Created by AI4Bharat, it provides benchmark datasets for 11 Indian languages including Tamil. We use the Tamil news headlines classification task from IndicGLUE.

Q17: Difference between BERT, XLM-RoBERTa, and MuRIL?

A: BERT: 104 languages, 110M params, trained on Wikipedia. XLM-RoBERTa: 100 languages, larger training data (2.5TB CommonCrawl), better training recipe. MuRIL: 17 Indian languages + English, uses transliterated text too, specifically optimized for India. MuRIL > XLM-R > BERT for Indian languages generally.

Q18: What is agglutinative language? Give example.

A: A language where complex words are formed by joining many morphemes. Tamil example: "?????????????" = ??? (read) + ??? (infinitive) + ????? (negation) = "did not read". One Tamil word = 3-4 English words. This is why character n-grams work - they capture these morpheme patterns.

Q19: Why is XLM-RoBERTa the "safest" choice?

A: Because it performs well on BOTH languages: 95.01% on Tamil and 82.61% on Bangla. No other model is consistently top-5 for both. It's the only model that doesn't fail for either language, making it the safest default when you don't know which model to pick.

Q20: What is your main conclusion in one sentence?

A: "There is no single best model for all Indian languages - the optimal choice depends on language family, dataset complexity, and available resources, with SVM best for simple Dravidian tasks and LLMs/Transformers best for complex Indo-Aryan tasks."

21. PRESENTATION FLOW (Slide Order)

1. Title Slide (Title, authors, guide, university)
 2. Problem Statement (Why Indian language NLP matters)
 3. Languages Studied (Tamil vs Bangla - families, properties)
 4. Datasets (IndicGLUE Tamil, AI4Bharat Bangla)
 5. Models Overview (20 models x 5 categories - taxonomy diagram)
 6. Architecture (System architecture diagram)
 7. Methodology (Feature extraction, training config)
 8. Tamil Results (Table + Bar chart)
 9. Bangla Results (Table + Bar chart)
 10. Cross-Language Comparison (Side-by-side chart)
 11. Key Findings (6-7 bullet points)
 12. Overfitting Analysis (Address SVM concerns)
 13. Practical Recommendations (Decision flowchart)
 14. Limitations (Be honest)
 15. Future Work (What's next)
 16. Conclusion (One slide, main takeaway)
 17. Thank You / Questions
-

22. ONE-PAGE SUMMARY (For quick revision)

- > Project: Compare 20 ML models for Tamil & Bangla text classification
- > Tamil: 5,346 samples, 3 classes -> SVM wins (95.43%)
- > Bangla: 20,000 samples, 6+ classes -> IndicGemma wins (84.82%)
- > Key insight: No universal best model - depends on language + dataset

- > Character n-grams: Secret weapon for agglutinative languages
 - > LLMs: Great for Bangla, terrible for Tamil (zero-shot)
 - > RNNs: Need 15+ epochs and 10K+ samples
 - > XLM-RoBERTa: Safest choice across both languages
 - > Caution: Tamil's high scores are partly because task is easy (3 clean classes)
 - > Main contribution: First 20-model cross-language comparison for Indian NLP
-

Study Guide Created: July 2026

Total Models: 20 | Languages: 2 | Categories: 5

Use this to prepare for presentation and viva questions.